

TECHNICAL BUILD SPECIFICATION · V2 — PRODUCTION / AWS

Assessment Portal Build & Deployment Spec

An SHL-style psychometric & skills assessment platform — architecture, data model, engine logic, and a production deployment plan on AWS, written to be executed by an agentic coding tool.

Prepared for: Ailexity Technology Pvt. Ltd.

Target executor: Claude Code (agentic build)

Stack: Next.js (App Router) · Prisma · PostgreSQL · Redis · Tailwind

Runtime: AWS — ECS Fargate · RDS · ElastiCache · S3 · behind ALB

Scope: Phase 1 MVP (build-ready) + AWS deployment + Phase 2–4 roadmap

1. Project Overview & Objective

Build a multi-tenant web application that lets organizations create timed assessments, invite candidates, deliver tests under strict server-enforced timing, and score results into reports. This document is written as an instruction set for Claude Code: it defines *what* to build and the *rules that must not be broken*, while leaving idiomatic implementation choices to the agent.

The product has two distinct front-ends over one backend:

- **Admin / Employer app** — authoring tests, managing question banks, inviting candidates, viewing reports.
- **Candidate app** — a focused, distraction-free, timed test-taking experience.

POSITIONING DECISION (DRIVES SCOPE)

Phase 1 is a **test-delivery SaaS**: the org supplies or builds its own questions, and we provide the engine plus a few starter banks. We are **not** claiming psychometric validity in Phase 1 — scores are labelled "screening-grade." True norming and validity are Phase 3+ concerns and must not block the MVP.

2. Tech Stack & Conventions

Layer	Choice	AWS managed service
Framework	Next.js 14+ (App Router, TypeScript)	Containerized → ECS Fargate
ORM / DB	Prisma + PostgreSQL	RDS for PostgreSQL (Multi-AZ)
Live state / timers	Redis	ElastiCache for Redis
Auth	Auth.js (admins) + tokenized links (candidates)	Secrets in Secrets Manager
Styling	Tailwind CSS	—
Object storage	S3 (question media, recordings)	Amazon S3 + CloudFront
Async scoring / email	Queue + worker	SQS → Fargate worker (or Lambda)
Validation	Zod on every API boundary	—

Non-negotiable engineering rules

1. **The server is the source of truth for time.** The client clock is display-only.
2. **Correct answers never reach the candidate client.** Scoring happens server-side only.
3. **Every response write is append-only and idempotent** (keyed by attemptId + questionId).
4. All money/PII/answer data access is scoped by `organizationId` — enforce multi-tenancy at the query layer.
5. **App servers are stateless.** No timer, session, or attempt state may live in a single instance's memory — it lives in ElastiCache/RDS so any of N Fargate tasks can serve any request. This is what makes the timer engine correct under horizontal scaling.

3. System Architecture

Seven logical modules. In Phase 1 they live inside one Next.js app; the session engine is the only part that may later be extracted into its own service.

Module	Responsibility
Question Bank	Stores items: type, stem, options, correct answer(s), media, difficulty, tags. The crown-jewel data.
Test Assembly	Builds a test from sections; supports fixed forms and randomized pools (pick N of M).
Session / Delivery Engine	Serves questions one at a time, enforces timers, auto-saves, handles resume/disconnect.
Scoring Engine	Responses → raw score → (Phase 3) percentile vs. norm group → apply cutoff.
Integrity Layer	Randomization, tab-switch flags, (Phase 3) webcam, time-anomaly detection.
Reporting	Candidate reports, section breakdowns, employer dashboards.
Invitations	Tokenized invite links, expiry, status tracking, bulk CSV import.

4. Data Model (Prisma Schema)

This is the Phase-1 schema. Claude Code should treat it as the starting migration and extend it per phase.

```
// schema.prisma — Phase 1

model Organization {
  id          String   @id @default(cuid())
  name        String
  users       User[]
  banks       QuestionBank[]
  tests       Test[]
  createdAt   DateTime @default(now())
}

model User {
  id          String   @id @default(cuid())
  email        String   @unique
  passwordHash String
  role         Role      @default(ADMIN)
  organizationId String
  organization Organization @relation(fields: [organizationId], references: [id])
  createdAt   DateTime @default(now())
}

enum Role { OWNER ADMIN VIEWER }

model QuestionBank {
  id          String   @id @default(cuid())
  name        String
  organizationId String
  organization Organization @relation(fields: [organizationId], references: [id])
  questions   Question[]
}

model Question {
  id          String   @id @default(cuid())
  bankId      String
  bank        QuestionBank @relation(fields: [bankId], references: [id])
  type        QType
  stem        String    // markdown supported
  mediaUrl    String?   // pre-rendered chart/image
  options     Option[]
  // correctOptionIds is server-only; never serialize to candidate
  difficulty  Float      @default(0.5) // provisional until calibrated
  tags        String[]
  createdAt   DateTime @default(now())
}

enum QType { MCQ_SINGLE MCQ_MULTI NUMERIC LIKERT }

model Option {
  id          String   @id @default(cuid())
  questionId  String
  question    Question @relation(fields: [questionId], references: [id])
  label       String
  isCorrect   Boolean @default(false) // NEVER sent to candidate client
  order       Int
}

model Test {
```

```

id          String    @id @default(cuid())
name        String
organizationId String
organization Organization @relation(fields: [organizationId], references: [id])
sections    Section[]
invitations Invitation[]
published   Boolean    @default(false)
createdAt   DateTime   @default(now())
}

model Section {
  id          String    @id @default(cuid())
  testId      String
  test        Test      @relation(fields: [testId], references: [id])
  name        String
  order       Int
  timeLimitSec Int
  questionCount Int      // how many to serve
  poolStrategy Pool      @default(FIXED)
  questionIds String[]   // pool to draw from
}

enum Pool { FIXED RANDOM_POOL }

model Invitation {
  id          String    @id @default(cuid())
  testId      String
  test        Test      @relation(fields: [testId], references: [id])
  email       String
  token       String    @unique           // used in the magic link
  status      InviteStatus @default(SENT)
  expiresAt   DateTime
  attempt     Attempt?
  createdAt   DateTime @default(now())
}

enum InviteStatus { SENT STARTED SUBMITTED EXPIRED }

model Attempt {
  id          String    @id @default(cuid())
  invitationId String    @unique
  invitation   Invitation @relation(fields: [invitationId], references: [id])
  startedAt   DateTime?
  submittedAt  DateTime?
  currentSection Int      @default(0)
  responses   Response[]
  score       Score?
  events      ProctoringEvent[]
}

model Response {
  id          String    @id @default(cuid())
  attemptId   String
  attempt     Attempt   @relation(fields: [attemptId], references: [id])
  questionId  String
  chosenOptionIds String[]
  numericValue Float?
  timeSpentMs Int

```

```

    answeredAt      DateTime @default(now())
    @@unique([attemptId, questionId]) // idempotent writes
}

model Score {
  id              String @id @default(cuid())
  attemptId       String @unique
  attempt         Attempt @relation(fields: [attemptId], references: [id])
  rawBySection    Json    // { sectionId: rawScore }
  rawTotal        Float
  percentile       Float? // Phase 3+
  passed          Boolean?
  computedAt      DateTime @default(now())
}

model ProctoringEvent {
  id              String @id @default(cuid())
  attemptId       String
  attempt         Attempt @relation(fields: [attemptId], references: [id])
  type            String // TAB_BLUR, PASTE, TIME_ANOMALY ...
  payload         Json?
  occurredAt      DateTime @default(now())
}

```

INDEX GUIDANCE

`Response` grows fastest — index on `attemptId`. Also index `Invitation.token` (already unique) and `Attempt.invitationId`. Treat `Response` as append-only.

5. The Timing / Session Engine

This is the component that most clones get wrong. The rules below are mandatory.

Authoritative timing rules

- On section start, write `sectionStartedAt` (server time) to Redis + DB. The countdown the candidate sees is derived from the server value on each request, never set by the client.
- On every question fetch and answer submit, the server recomputes elapsed time. If the section window has closed, the server rejects further answers for that section and auto-advances.
- No pausing within a section. A short break between sections is allowed only if configured.
- On refresh/reconnect: resume from last saved state; **the timer keeps running** (it is wall-clock from `sectionStartedAt`, not paused).
- Ignore any client-supplied timestamp for scoring or gating. `timeSpentMs` from the client is stored for analytics only, never trusted for enforcement.

WHY THIS MATTERS ON AWS

Behind an Application Load Balancer, consecutive requests from one candidate can land on different Fargate tasks. Because timing is derived from `sectionStartedAt` stored in **ElastiCache** (not instance memory), the countdown is identical regardless of which task answers. Do *not* use sticky sessions to paper over in-memory state — keep the tasks stateless instead. Use a short Redis lock (`SET NX PX`) around the answer→cursor update to prevent double-submit races when a candidate double-clicks.

Session state (ElastiCache Redis) — pseudocode

```
// key: attempt:{attemptId}:section:{idx}
{
  sectionStartedAt: 1737100000000,    // server epoch ms
  timeLimitSec: 1080,
  servedQuestionIds: ["q1","q7",...], // fixed order for this attempt
  cursor: 3                          // index of next question
}

// GET /api/attempt/:id/next
function nextQuestion(attemptId) {
  s = redis.get(sessionKey)
  elapsed = now() - s.sectionStartedAt
  if (elapsed >= s.timeLimitSec * 1000) return advanceSection(attemptId)
  q = loadQuestionForCandidate(s.servedQuestionIds[s.cursor]) // strips isCorrect
  return { question: q, remainingMs: s.timeLimitSec*1000 - elapsed }
}

// POST /api/attempt/:id/answer (idempotent on attemptId+questionId)
function submitAnswer(attemptId, questionId, payload) {
  s = redis.get(sessionKey)
  if (now() - s.sectionStartedAt >= s.timeLimitSec*1000)
    return { rejected: "SECTION_EXPIRED" }
  upsertResponse(attemptId, questionId, payload) // append-only upsert
  s.cursor++ ; redis.set(sessionKey, s)
  if (s.cursor >= s.servedQuestionIds.length) return advanceSection(attemptId)
  return { ok: true }
}
```

6. Scoring Engine

Phase 1 scoring is deliberately simple and honest.

- **Raw scoring:** compare stored `chosenOptionIds` against server-held correct options per question; sum per section into `rawBySection`.
- **No negative marking** by default (candidates should be told this so they always guess).
- **Cutoff:** a simple raw-score or raw-percentage threshold per test → sets `passed`.
- **Percentile (Phase 3):** only once a norm group of real responses exists. Until then, `percentile` stays null and reports say "screening-grade, un-normed."
- Scoring runs on submit *and* is re-runnable (idempotent) so rescoring after a key fix is safe.

HONESTY GUARDRAIL

Do not display "percentile" or "compared to X candidates" language anywhere until a real norm group exists. Mislabelling un-normed scores as percentiles is the fastest way to lose enterprise credibility (and creates legal exposure in hiring).

7. API Surface (Phase 1)

Method & Route	Purpose	Auth
POST /api/auth/*	Admin login/session	—
POST /api/banks / GET	Create/list question banks	Admin
POST /api/questions	Add question (+options, media)	Admin
POST /api/tests / PATCH	Build/publish test & sections	Admin
POST /api/invitations	Create invites (single + CSV bulk)	Admin
GET /api/take/:token	Resolve invite → start/resume attempt	Token
GET /api/attempt/:id/next	Serve next question + remaining time	Token
POST /api/attempt/:id/answer	Submit answer (idempotent)	Token
POST /api/attempt/:id/event	Log proctoring event (tab blur, paste)	Token
POST /api/attempt/:id/submit	Finalize + trigger scoring	Token
GET /api/tests/:id/results	Employer results + report data	Admin
GET /api/health	Liveness/readiness for ALB target group	—

8. Suggested File Structure

```
/app
/(admin)                # authed employer app
  /dashboard/page.tsx
  /banks/[id]/page.tsx
  /tests/[id]/builder/page.tsx
  /tests/[id]/results/page.tsx
/take/[token]/page.tsx  # candidate full-screen runner
/api
  /banks/route.ts
  /questions/route.ts
  /tests/route.ts
  /invitations/route.ts
  /attempt/[id]/next/route.ts
  /attempt/[id]/answer/route.ts
  /attempt/[id]/submit/route.ts
/lib
  /session-engine.ts      # Redis timer logic (Section 5)
  /scoring.ts             # raw scoring (Section 6)
  /assembly.ts            # pool selection, question ordering
  /candidate-serializer.ts # STRIPS isCorrect before send
  /auth.ts /db.ts /redis.ts
/prisma/schema.prisma
/components/ (admin + runner UI)
```

CRITICAL HELPER

`candidate-serializer.ts` is a security boundary. Every question object sent to the candidate must pass through it so `isCorrect` and difficulty metadata can never leak via network inspection.

9. AWS Deployment Architecture

The application is packaged as a Docker image and run as an auto-scaling ECS Fargate service behind an Application Load Balancer, with managed data stores. No servers to patch; everything scales on load.

Request path (top to bottom)

```
Route 53 (DNS)
|
CloudFront — static assets & S3 media (cached at edge)
|
AWS WAF — rate limits, bot/OWASP rules
|
Application Load Balancer (HTTPS, ACM cert)
|   target group, health check GET /api/health
▼
ECS Fargate service (Next.js container, 2..N tasks, auto-scaled)
├─▶ RDS PostgreSQL (Multi-AZ, private subnet)
├─▶ ElastiCache Redis (timers, locks, private subnet)
├─▶ S3 (question media, proctoring recordings)
└─▶ SQS → Worker task (async scoring, invite emails via SES)
```

Service-by-service mapping

Concern	AWS service	Config notes
Compute	ECS on Fargate	Stateless tasks; min 2 for availability; behind private subnets.
Ingress / TLS	ALB + ACM	HTTPS only; HTTP→HTTPS redirect; health check <code>/api/health</code> .
Edge / CDN	CloudFront	Serves Next.js static output and S3 media; long cache on hashed assets.
WAF	AWS WAF on ALB/ CloudFront	Rate-based rules on <code>/api/take</code> & <code>/api/attempt/*</code> ; managed OWASP set.
Primary DB	RDS PostgreSQL	Multi-AZ, automated backups, 7–30d retention, encryption at rest (KMS).
Timer / cache	ElastiCache Redis	Single primary + replica; used for <code>sectionStartedAt</code> , locks, rate limit counters.
Media / recordings	S3	Private buckets; access via pre-signed URLs only; lifecycle rules for recordings.
Async work	SQS + worker	Scoring and email decoupled from the request; retries + dead-letter queue.
Email	SES	Invitation + verification emails; DKIM/SPF configured.
Container registry	ECR	Image scanning on push.
Secrets	Secrets Manager / SSM	DB URL, Redis URL, auth secrets injected as task env — never in the image.
Registry of images	ECR	Immutable tags per release (git SHA).

10. Networking & Security

VPC layout

- **Public subnets:** ALB + NAT gateway only.
- **Private subnets:** Fargate tasks, RDS, ElastiCache — no public IPs.
- Fargate reaches the internet (SES, S3) via NAT / VPC endpoints; RDS and Redis are never internet-facing.

Security groups (least privilege)

From → To	Port
Internet → ALB	443
ALB → Fargate tasks	app port (e.g. 3000)
Fargate → RDS	5432
Fargate → ElastiCache	6379

- Encryption in transit (TLS) and at rest (KMS) on RDS, ElastiCache, S3.
- IAM task roles scoped to exactly the S3 prefix / SQS queue / secret each task needs.
- S3 buckets: block all public access; serve media only through pre-signed URLs or CloudFront OAC.
- Prisma connection pooling via RDS Proxy (or PgBouncer) so scaling tasks don't exhaust DB connections.

DPDP NOTE (INDIA)

Keep candidate PII and proctoring recordings in an AWS region appropriate to your data-residency commitments (e.g. `ap-south-1`, Mumbai). Apply S3 lifecycle rules to auto-expire recordings after your stated retention window, and make deletion requests actionable.

11. Scaling, Availability & the Timer Engine

- **Auto-scaling:** scale Fargate task count on CPU/ALB request-count target tracking. Assessments are bursty (a whole cohort starts at 10:00 AM) — set a sensible max and consider scheduled scale-out before known test windows.
- **Statelessness is the enabler:** because timers live in ElastiCache and responses in RDS, adding/removing tasks never affects an in-progress attempt.
- **Multi-AZ everywhere:** Fargate across ≥ 2 AZs, RDS Multi-AZ, ElastiCache with a replica. Loss of one AZ should not fail active attempts.
- **Graceful deploys:** rolling ECS deployment with connection draining on the ALB so a deploy mid-test doesn't drop a candidate; the stateless design means a task can be replaced without losing attempt state.
- **Idempotency + locks** (from Section 5) protect the answer path during scale events and retries.

12. CI/CD & Infrastructure as Code

Pipeline

```
GitHub push (main)
→ GitHub Actions
  1. install, typecheck, test
  2. docker build → push to ECR (tag = git SHA)
  3. run prisma migrate deploy (against RDS, gated)
  4. update ECS service (new task def) → rolling deploy
  5. smoke test /api/health
```

- **IaC:** define all of the above in Terraform (or AWS CDK). Claude Code should generate a `/infra` directory — VPC, subnets, SGs, RDS, ElastiCache, ECS service, ALB, S3, SQS, IAM roles — so environments are reproducible.
- **Environments:** at least `staging` and `prod` with separate state and secrets.
- **Migrations:** `prisma migrate deploy` runs as a one-off ECS task (or pipeline step) before the new app version takes traffic — never auto-migrate on container boot across N tasks.
- **Config:** all runtime config via env vars sourced from Secrets Manager; the image contains no secrets and is environment-agnostic.

Dockerfile expectations

- Multi-stage build; Next.js `output: "standalone"` for a small runtime image.
- Non-root user; only the standalone server + static assets in the final stage.
- Expose the app port; container answers `/api/health` for ALB checks.

13. Observability & Backups

- **Logs:** container logs → CloudWatch Logs; structured JSON so attempts/anomalies are queryable.
- **Metrics & alarms:** CloudWatch alarms on 5xx rate, ALB latency, Fargate CPU/mem, RDS connections/CPU, Redis evictions. Page on error-rate spikes during live test windows.
- **Tracing (optional):** AWS X-Ray or OpenTelemetry on the answer/scoring path.
- **Backups:** RDS automated backups + periodic manual snapshots; test a restore. ElastiCache is a cache, not a system of record — an attempt's durable truth (responses, `startedAt`) is also persisted to RDS so a Redis flush degrades gracefully rather than losing progress.
- **DR posture:** document RPO/RTO; snapshots + IaC allow rebuild in a second region if needed.

REDIS DURABILITY CAVEAT

Timers are convenience state in Redis, but the authoritative `sectionStartedAt` and every `Response` are also written to RDS. On a Redis failure the session engine rehydrates from RDS. Never let Redis be the *only* place an in-progress attempt exists.

14. Rough Cost Envelope (early production)

Order-of-magnitude only (ap-south-1, low-but-real traffic); tune to actual load.

Service	Indicative monthly
ECS Fargate (2 small tasks baseline)	\$30–70
RDS PostgreSQL (Multi-AZ, small)	\$60–120
ElastiCache Redis (small + replica)	\$30–60
ALB	\$18–30
S3 + CloudFront + data transfer	\$5–30 (usage)
NAT gateway	\$32+ (often the sneaky line item)
SES / SQS / CloudWatch / Secrets	\$5–20

Cost-down for the earliest stage: a single-AZ RDS, one NAT (or VPC endpoints to avoid NAT), and Fargate min=1 on staging. Move to Multi-AZ before real client cohorts run.

15. Phased Roadmap

Phase 1 MVP — build now (~6–8 weeks)

- Org accounts + admin auth (multi-tenant).
- Question bank: MCQ (single/multi) + numeric + image media.
- Fixed-form test builder with sections & per-section timers.
- Tokenized invitations (single + CSV bulk).
- Candidate runner: full-screen, server-timed delivery, auto-save, resume.
- Raw scoring + simple cutoff + basic candidate/employer report.

Phase 2 Scale the engine

- Randomized question pools (pick N of M) + per-attempt ordering.
- Likert personality & situational-judgment item types.
- Employer analytics dashboard; basic proctoring (tab-switch / paste flags).

Phase 3 Credibility layer

- Norming: build distributions from accumulated responses → real percentiles.
- Webcam/screen proctoring with anomaly flagging; two-stage verification re-test.
- Provisional reliability reporting (Cronbach's alpha per section).

Phase 4 Enterprise

- Adaptive testing (requires IRT-calibrated items — Python scoring service).

- ATS integration API + white-labelling.

16. Phase-1 Acceptance Criteria

Claude Code should treat these as the definition of done for the MVP:

1. An admin can create a bank, add 20+ questions, and assemble a 2-section timed test.
2. A candidate opening an invite link can complete the test; refreshing mid-section resumes without gaining extra time.
3. Submitting an expired section is rejected server-side even with a forged client clock.
4. No network response to the candidate ever contains `isCorrect`.
5. On submit, a raw score with section breakdown appears in the employer results view.
6. Tab-blur events are recorded against the attempt.
7. All queries are scoped by `organizationId`; no cross-tenant data access is possible.
8. The app runs as a stateless container: two tasks behind an ALB can each serve the same candidate's consecutive requests with an identical countdown (no sticky sessions).
9. Killing/replacing one Fargate task mid-attempt does not lose progress or reset the timer.
10. `GET /api/health` returns 200 for ALB checks; secrets are injected from Secrets Manager, absent from the image.
11. The full stack is reproducible from the `/infra` IaC in a fresh account/region.

17. Compliance & Risk Notes

- **India DPDP Act:** candidate data is sensitive PII — capture consent, state purpose & retention, secure any recordings, and support deletion.
- **Item security:** question banks are IP and leak easily; pools + randomization + serializer stripping reduce (not eliminate) scraping.
- **Bias / adverse impact:** once used for real hiring, adverse-impact analysis is not optional — it protects clients and your reputation. Scope it into Phase 3.
- **Do not overclaim validity.** Until Phase 3 norming, label everything "screening-grade."